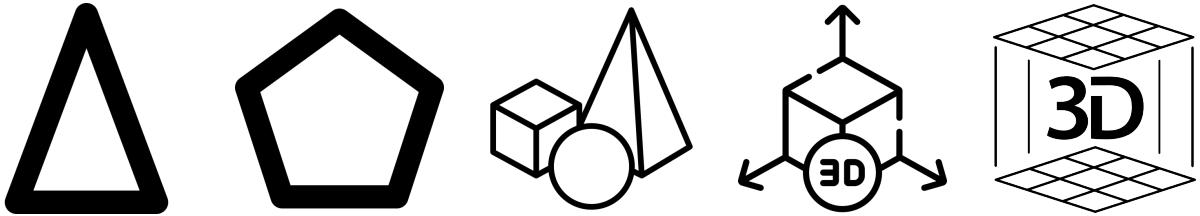


CSC 4505 (2025/26)
Graphics and Visual Computing

Lecture 5: Vertex Specification

Lighton Phiri <lighton.phiri@unza.zm>
Department of Computing and Informatics
University of Zambia
https://datalab.unza.zm/people/lightonphiri

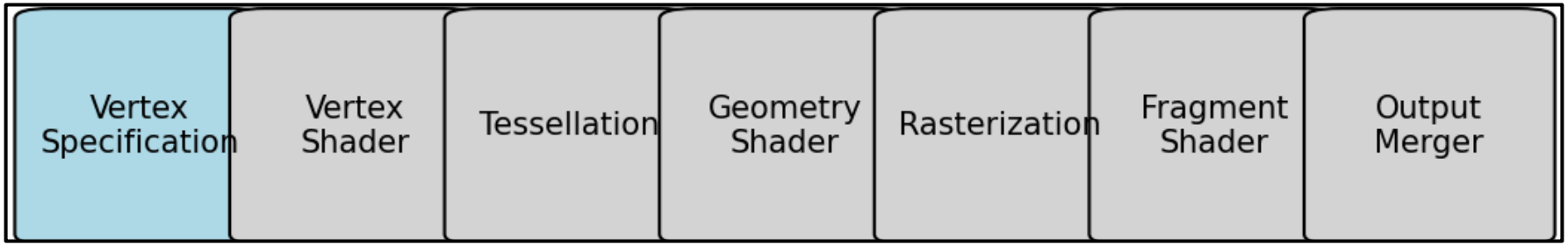
Lecture Series Outline

- Recap – The Graphics Pipeline
- Vertex Attributes – What Makes Up a Vertex?
- Understanding the Coordinate System on the Pygame Canvas
- Specifying 3D Vertices in OpenGL (Immediate Mode)
- Drawing Primitives – Points, Lines, and Triangles
- Building Complex Objects – A 3D Cube
- Complete 3D Example with Code Explanation
- Next Steps in the Pipeline
- Code Repository and File List

Lecture Series Outline

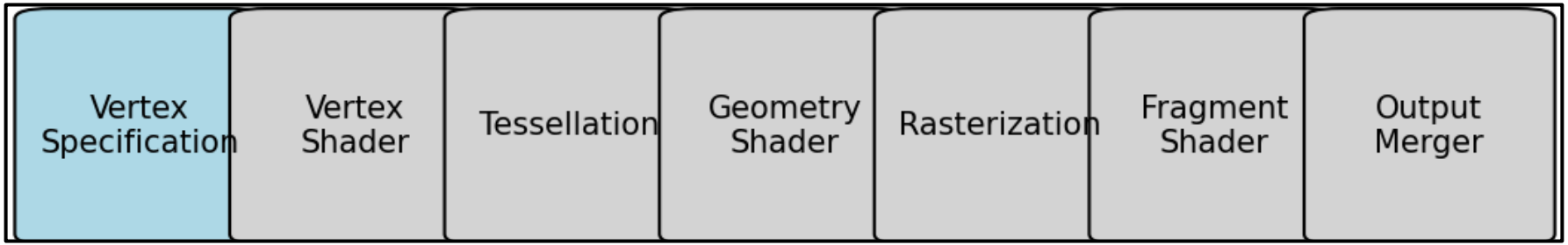
- Recap – The Graphics Pipeline
- Vertex Attributes – What Makes Up a Vertex?
- Understanding the Coordinate System on the Pygame Canvas
- Specifying 3D Vertices in OpenGL (Immediate Mode)
- Drawing Primitives – Points, Lines, and Triangles
- Building Complex Objects – A 3D Cube
- Complete 3D Example with Code Explanation
- Next Steps in the Pipeline
- Code Repository and File List

Where Are We in the Pipeline? (1/4)



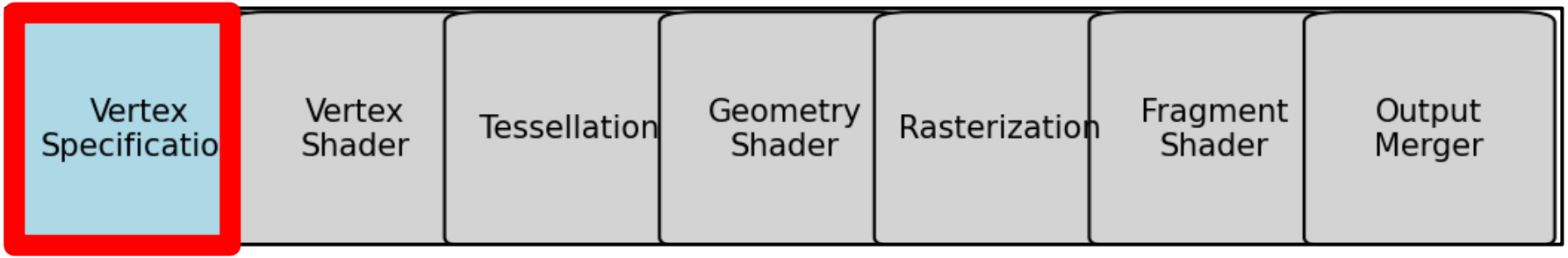
- **The graphics pipeline transforms 3D data into 2D images**
  - Vertex Specification – We provide raw vertex data (positions, colors, etc.) to the GPU
  - Vertex Shader – A small program that processes each vertex individually
  - Tessellation (optional) – Subdivides primitives
  - Geometry Shader (optional) – Can add or remove primitives

Where Are We in the Pipeline? (2/4)



- **The graphics pipeline transforms 3D data into 2D images**
  - Rasterization – Converts primitives into fragments (potential pixels)
  - Fragment Shader – Colors each fragment
  - Output Merging – Combines fragments with the framebuffer

Where Are We in the Pipeline? (3/4)



- **The graphics pipeline transforms 3D data into 2D images**
  - Vertex Specification is the very first stage: we provide raw vertex data to the GPU
- **In this lecture series, we will feed vertices directly in NDC – range [-1,1] on all axes**
- **Later lectures will add transformations to move objects through world, view, and clip spaces**

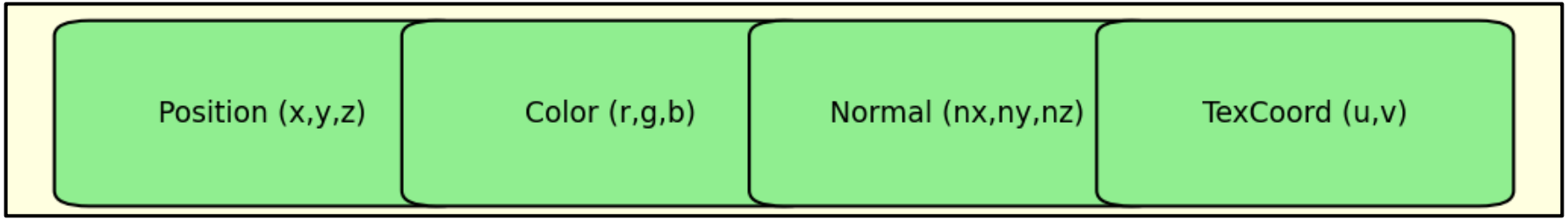
Where Are We in the Pipeline? (4/4)

- **The graphics pipeline transforms 3D data into 2D images**
  - Vertex Specification is the very first stage: we provide raw vertex data to the GPU
- **In this lecture series, we will feed vertices directly in NDC – range [-1,1] on all axes**
- **Later lectures will add transformations to move objects through world, view, and clip spaces**

Lecture Series Outline

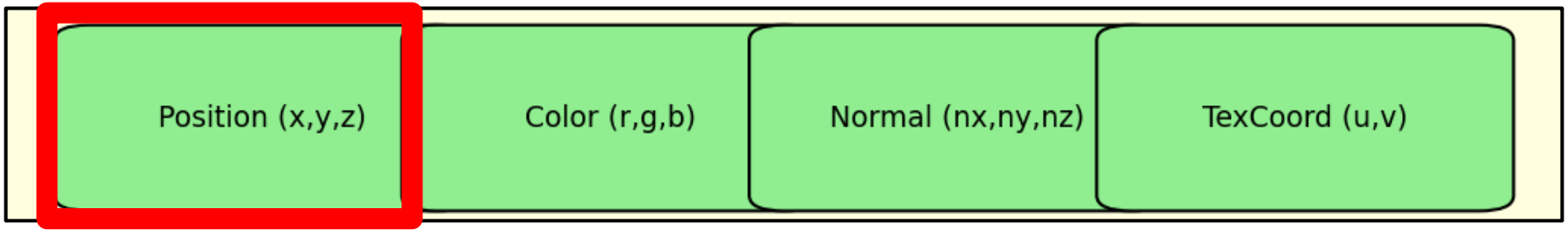
- Recap – The Graphics Pipeline
- Vertex Attributes – What Makes Up a Vertex?
- Understanding the Coordinate System on the Pygame Canvas
- Specifying 3D Vertices in OpenGL (Immediate Mode)
- Drawing Primitives – Points, Lines, and Triangles
- Building Complex Objects – A 3D Cube
- Complete 3D Example with Code Explanation
- Next Steps in the Pipeline
- Code Repository and File List

What is a Vertex?



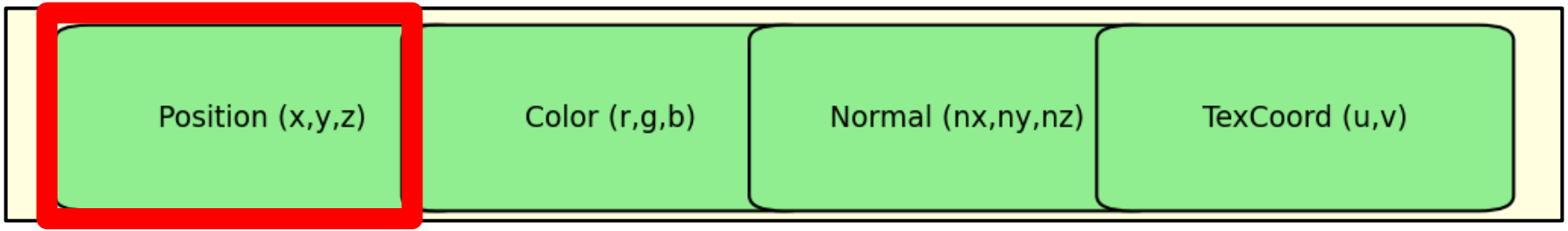
- In everyday language, a vertex is a corner point of a shape
- In computer graphics, a vertex is much richer: it is a collection of data associated with that point
- Each piece of data is called an attribute
- Attributes are inputs to the vertex shader and can be used for many purposes: positioning, coloring, lighting, texturing, animation, etc.

Vertex Attribute: Position (1/3)



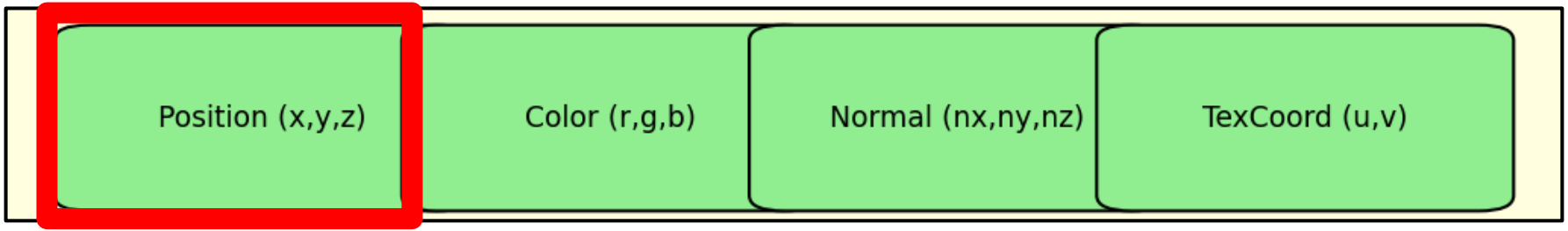
- **Position is the only mandatory attribute**
- **Usually represented as 3 components (x, y, z) or 4 components (x, y, z, w)**
- **What are x, y, z?**
  - They are coordinates in some space (initially NDC, later object/world/view/clip space)

Vertex Attribute: Position (2/3)



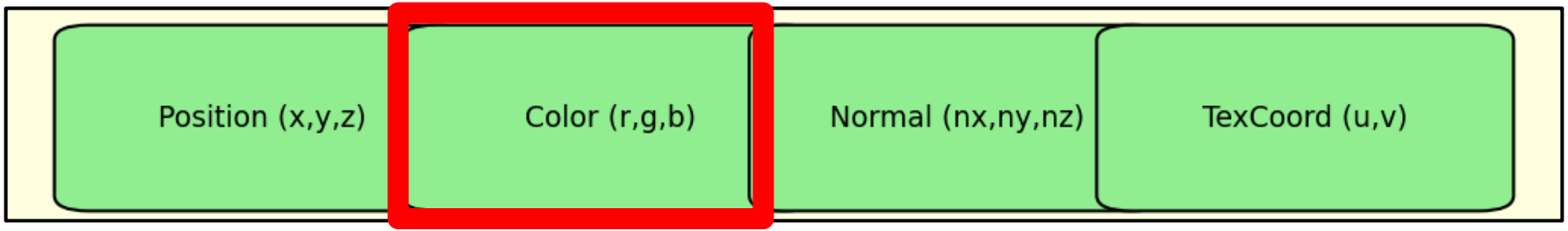
- **What are x, y, z?**
  - They are coordinates in some space (initially NDC, later object/world/view/clip space)
  - x: horizontal axis (left/right)
  - y: vertical axis (up/down)
  - z: depth axis (near/far)

Vertex Attribute: Position (3/3)



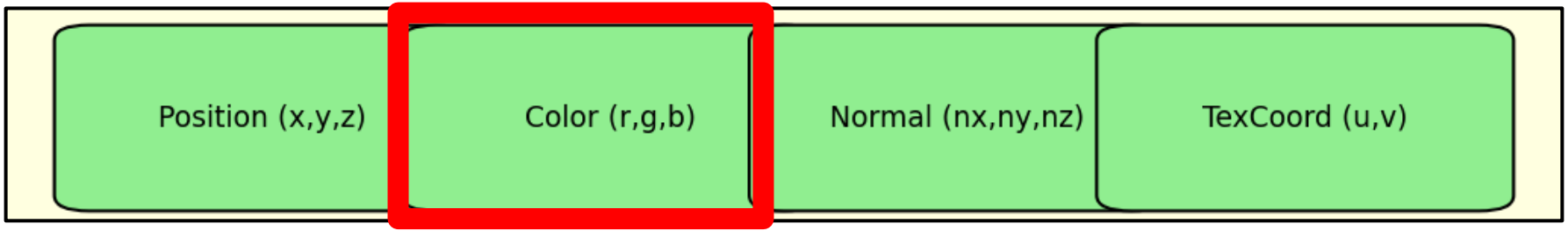
- **What is w?**
  - In homogeneous coordinates, w is a scaling factor
  - For Cartesian points, we typically set w = 1
  - Allows translation to be represented as matrix multiplication (important for transformations)
  - In clip space, w can be used for perspective division
- **Without position, GPU does not know where to place vertex**

Vertex Attribute: Color (1/6)



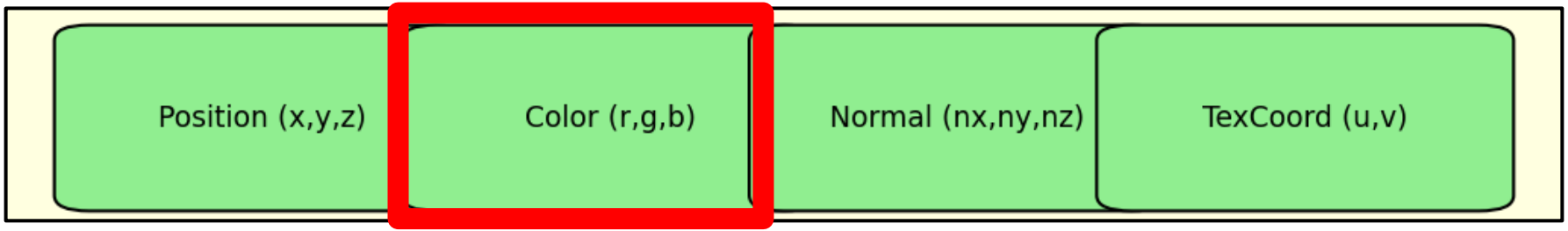
- **Color is optional but very common**
- **Usually represented as 3 components (r, g, b) or 4 components (r, g, b, a)**
- **What are r, g, b?**
  - Red, green, blue intensity values; combined to form any color
  - Typically in range [0.0, 1.0] for floating point, or [0, 255] for unsigned bytes

Vertex Attribute: Color (2/6)



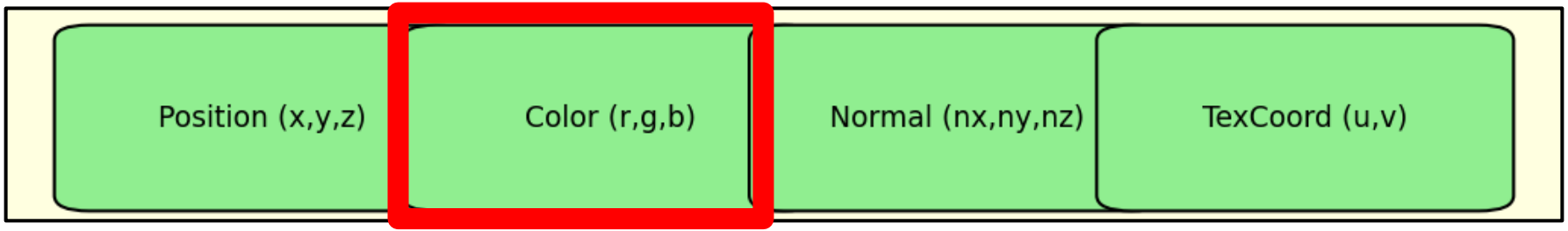
- **What is a (alpha)?**
  - Alpha channel: transparency
  - 0.0 = fully transparent, 1.0 = fully opaque
  - Used for blending, translucency, etc

Vertex Attribute: Color (3/6)



- **Why is "Color" important?**
  - Provides per-vertex color that can be interpolated across the triangle
  - Enables smooth color gradients (e.g., red to green)
  - Base for many shading effects

Vertex Attribute: Color (4/6)



- **Each component is a floating-point value between 0.0 and 1.0.**
  - 0.0 means no contribution of that colour
  - 1.0 means maximum intensity

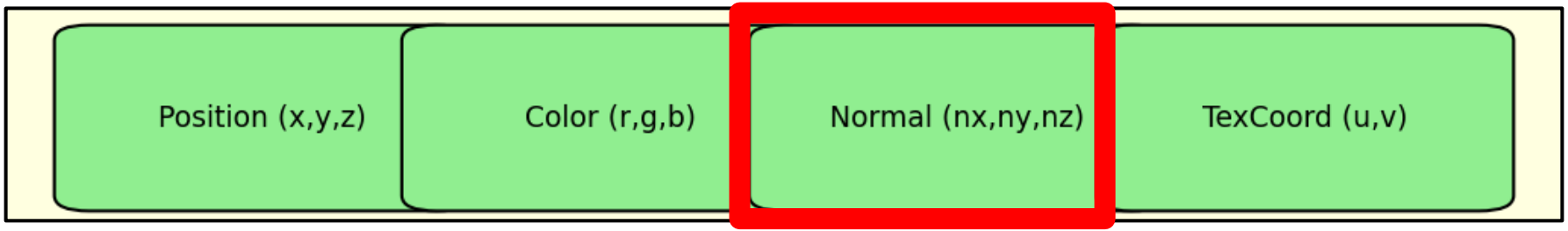
Vertex Attribute: Color (5/6)

- **Common colour examples**
  - Red: (1.0, 0.0, 0.0)
  - Green: (0.0, 1.0, 0.0)
  - Blue: (0.0, 0.0, 1.0)
  - Yellow: (1.0, 1.0, 0.0) (red + green)
  - Cyan: (0.0, 1.0, 1.0) (green + blue)
  - Magenta:(1.0, 0.0, 1.0) (red + blue)
  - White: (1.0, 1.0, 1.0)
  - Black: (0.0, 0.0, 0.0)

Vertex Attribute: Color (6/6)

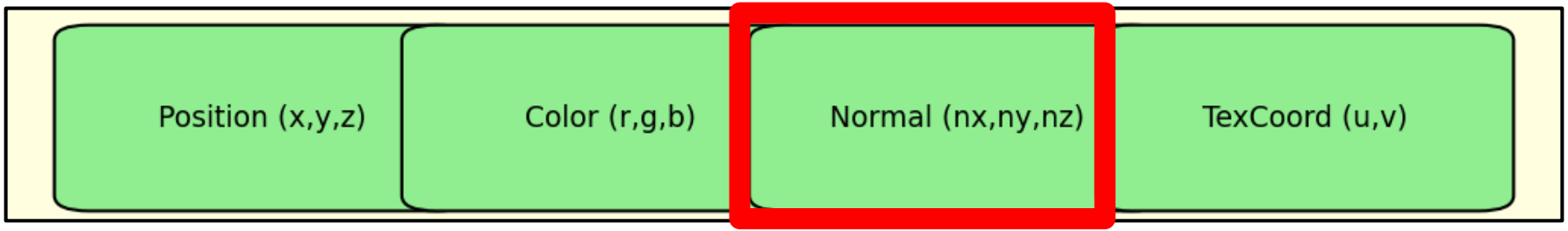
- In OpenGL immediate mode, we set the current colour with glColor3f(r, g, b)
- All vertices drawn after this call will use that colour until we change it

Vertex Attribute: Normal (1/3)



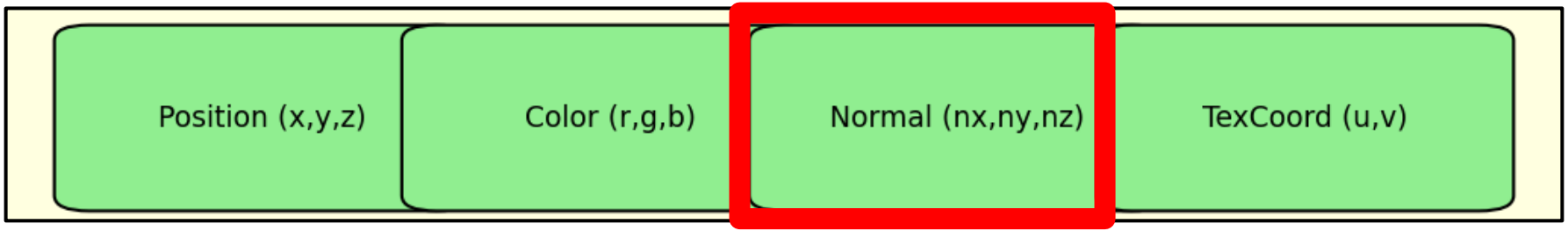
- **Normal is optional but essential for lighting.**
- **Represented as 3 components (nx, ny, nz).**
- **What is a normal vector?**
  - A unit vector (length = 1) perpendicular to the surface at the vertex
  - Points outward from the surface

Vertex Attribute: Normal (2/3)



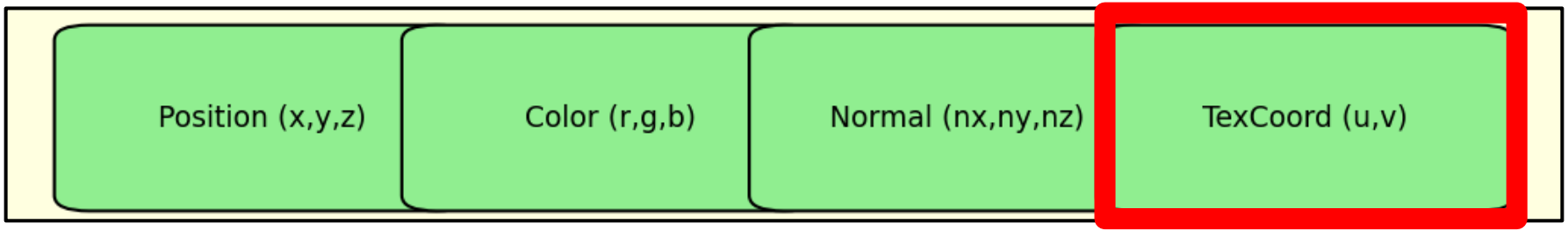
- **Why is the "Normal" important?**
  - Lighting calculations depend on the angle between the light direction and the normal
  - Example: If light hits a surface straight on (angle 0°), it's brightest; if it grazes the surface (90°), it's dark
  - Without normals, we cannot do realistic shading (flat shading would look unnatural)

Vertex Attribute: Normal (3/3)



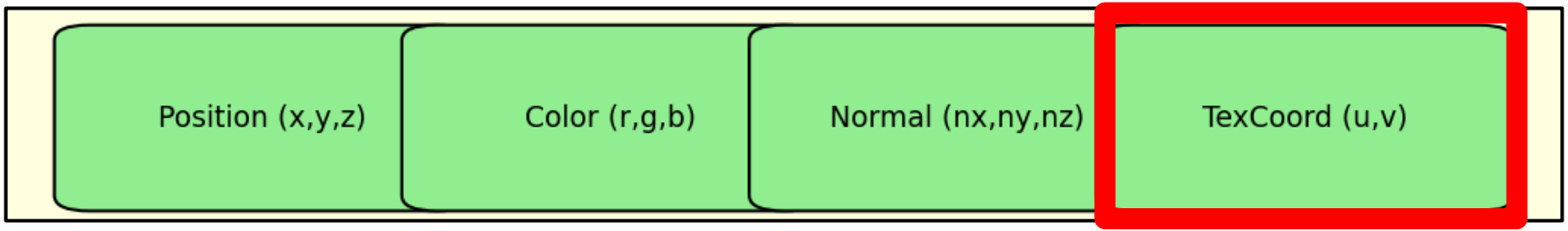
- **Why is the "Normal" important?**
  - Normals are interpolated across the triangle for smooth lighting (Phong shading)

Vertex Attribute: Texture Coordinates (1/3)



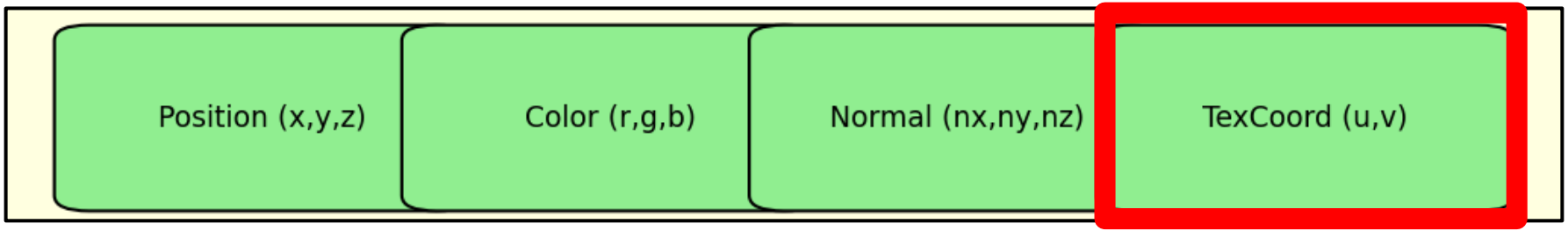
- Texture coordinates (also called UVs) are optional but essential for texturing
- Usually represented as 2 components (u, v); sometimes 3 for 3D textures

Vertex Attribute: Texture Coordinates (2/3)



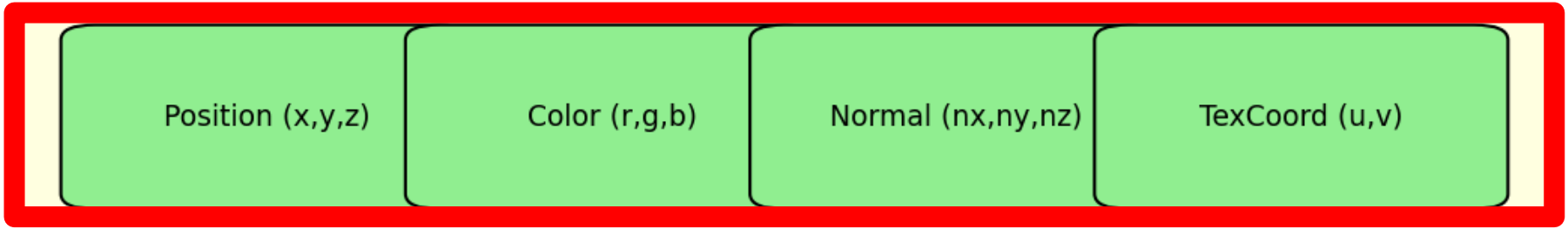
- **What are u and v?**
  - They are coordinates in a 2D texture image
  - u = horizontal coordinate (0 = left edge, 1 = right edge)
  - v = vertical coordinate (0 = bottom edge, 1 = top edge)
    - Note: In some graphics systems, v=0 is top, but OpenGL uses bottom as 0 by default; it can be flipped

Vertex Attribute: Texture Coordinates (3/3)



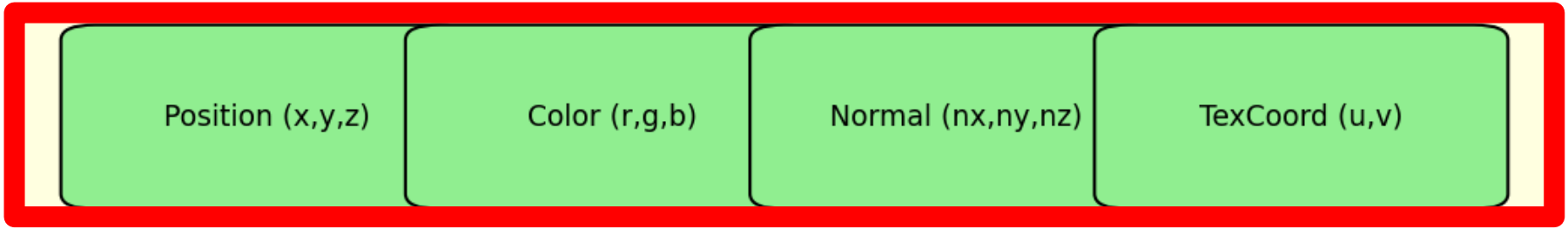
- **Why are "Texture Coordinates" important?**
  - They map a 2D image (texture) onto a 3D surface
  - Each vertex gets a (u,v) pair; during rasterization, these are interpolated so every fragment knows which texel (texture pixel) to sample
  - Enables detailed surface appearance without adding more geometry (e.g., brick wall, grass, skin)

Vertex Attribute: Custom Attributes



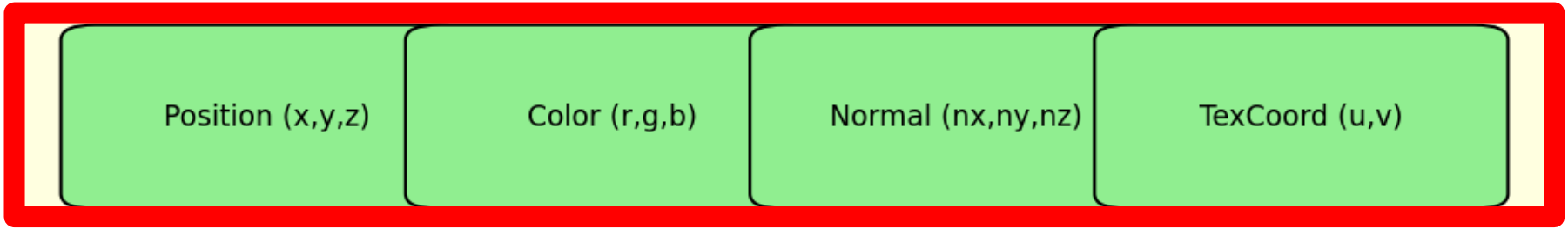
- **OpenGL allows you to define any number of attributes (up to a hardware limit, typically ≥16)**
  - Custom attributes give enormous flexibility
- **Some Examples**
  - Bone weights and bone indices for skeletal animation (skinning)
  - Tangents and bitangents for normal mapping
  - Vertex IDs for picking or special effects

Vertex Attribute: Attribute Locations (1/2)



- **In the vertex shader, each input attribute is identified by a location index**
  - We assign these indices when setting up attribute pointers
  - The shader declares: layout(location = 0) in vec3 aPos;
  - This connects the data in the VBO to the shader variable

Vertex Attribute: Attribute Locations (2/2)



- **Example mapping**
  - location 0 = position
  - location 1 = color
  - location 2 = normal
  - location 3 = texcoord

Lecture Series Outline

- Recap – The Graphics Pipeline
- Vertex Attributes – What Makes Up a Vertex?
- Understanding the Coordinate System on the Pygame Canvas
- Specifying 3D Vertices in OpenGL (Immediate Mode)
- Drawing Primitives – Points, Lines, and Triangles
- Building Complex Objects – A 3D Cube
- Complete 3D Example with Code Explanation
- Next Steps in the Pipeline
- Code Repository and File List

Understanding the Pygame Canvas (1/2)

- **When we draw using NDC coordinates, the GPU automatically maps the NDC cube to the viewport (the Pygame window)**
- **This mapping is called the viewport transformation**
- **For a window of size 800×600**
  - NDC x = -1 maps to screen x = 0
  - NDC x = +1 maps to screen x = 800
  - Similarly for y (with possible inversion depending on coordinate system)

Understanding the Pygame Canvas (2/2)

- Important: The coordinates we specify are in model space (for now, we place objects directly in NDC). The GPU transforms them through the pipeline; by the time they appear on the canvas, they have been rasterised and are in screen space
- In immediate mode, we are effectively providing vertices in NDC, and OpenGL handles the rest

Lecture Series Outline

- Recap – The Graphics Pipeline
- Vertex Attributes – What Makes Up a Vertex?
- Understanding the Coordinate System on the Pygame Canvas
- Specifying 3D Vertices in OpenGL (Immediate Mode)
- Drawing Primitives – Points, Lines, and Triangles
- Building Complex Objects – A 3D Cube
- Complete 3D Example with Code Explanation
- Next Steps in the Pipeline
- Code Repository and File List

Drawing Primitives: Points, Lines, Triangles

- Now that we understand what a vertex is, we need to understand what we can draw with them.
- In OpenGL, vertices are assembled into primitives: points, lines, and triangles.
- The type of primitive is specified when we issue a drawing command.

Drawing Primitives: Immediate Mode—The Simplest Way to Draw (1/4)

- Before diving into modern OpenGL, we'll start with immediate mode (legacy OpenGL)
- Immediate mode uses glBegin() and glEnd() to specify vertices directly
- Why start here? It makes the concept of vertex specification concrete without the complexity of buffers and shaders.

Drawing Primitives: Immediate Mode—The Simplest Way to Draw (2/4)

- glBegin(primitive_type) marks the start of a list of vertices that will form one or more primitives
- The primitive_type argument tells OpenGL how to interpret the vertices that follow
- After glBegin, we call glVertex* functions to specify vertices

Drawing Primitives: Immediate Mode—The Simplest Way to Draw (3/4)

- Common primitive types

Module	Contents
GL_POINTS	Each vertex is a point
GL_LINES	Each pair of vertices forms a line segment
GL_LINE_STRIP	Vertices form a connected polyline
GL_LINE_LOOP	Similar to strip but connects last vertex back to first
GL_TRIANGLES	Each triple of vertices forms a triangle
GL_TRIANGLE_STRIP	Vertices form a strip of triangles
GL_TRIANGLE_FAN	First vertex is hub, subsequent pairs form triangles

Drawing Primitives: Immediate Mode—The Simplest Way to Draw (4/4)

- glEnd() marks the end of a vertex list
- It signals that all vertices for the current primitive have been specified
- Every glBegin must be matched with a glEnd.
- Between them, only certain OpenGL functions are allowed: glVertex, glColor, glNormal, glTexCoord, etc
- Calling other functions (like matrix operations) between glBegin and glEnd is illegal and may cause errors

Drawing Primitives: The glVertex Family of Functions (1/2)

- **OpenGL provides many variants of glVertex to handle different data types and dimensions**
- **Naming convention: glVertex{2|3}{s|i|f|d}{v}**
  - 2 or 3 – number of dimensions (x,y) or (x,y,z)
  - s – short integer (16-bit)
  - i – integer (32-bit)
  - f – float (32-bit)
  - d – double (64-bit)
  - v – vector (pointer to an array) – the "v" suffix

Drawing Primitives: The glVertex Family of Functions (2/2)

- **OpenGL provides many variants of glVertex to handle different data types and dimensions**
- **Naming convention: glVertex{2|3}{s|i|f|d}{v}**
  - 2 or 3 – number of dimensions (x,y) or (x,y,z)
  - s – short integer (16-bit)
  - i – integer (32-bit)
  - f – float (32-bit)
  - d – double (64-bit)
  - v – vector (pointer to an array) – the "v" suffix

Drawing Primitives: Commonly Used glVertex Variants (with examples) (1/3)

- **glVertex2f(x, y)**
  - 2D vertex using floats
  - glVertex2f(0.5, -0.5)
- **glVertex3f(x, y, z)**
  - 3D vertex using floats
  - glVertex3f(0.0, 0.0, 1.0)
- **glVertex2i(x, y)**
  - 2D vertex using integers
  - glVertex2i(100, 200)
  - Note NDC expects [-1,1]

Drawing Primitives: Commonly Used glVertex Variants (with examples) (2/3)

- **glVertex3d(x, y, z)**
  - 3D vertex using doubles
  - glVertex3d(0.5, -0.2, 0.0)
- **glVertex2fv([x, y])**
  - 2D vertex from a list/tuple of floats
  - glVertex2fv([0.5, -0.5])
- **glVertex3fv([x, y, z])**
  - 3D vertex from a list/tuple
  - glVertex3fv([0.0, 0.0, 1.0])

Drawing Primitives: Commonly Used glVertex Variants (with examples) (3/3)

- **glVertex2iv([x, y])**
  - 2D vertex from integer list
  - glVertex2iv([100, 200])
- **glVertex3sv([x, y, z])**
  - 3D vertex from short integer list (rarely used)
- **The vector versions (glVertex* v) are useful when vertex coordinates are stored in arrays**

Drawing Primitives: Why Multiple Data Types?

- **Precision**
  - Use float (or double) for world coordinates, int or short for screen-space coordinates in some contexts
- **Memory**
  - Integers and shorts consume less memory than floats, but may require conversion
- **In immediate mode, OpenGL converts all types to floats internally anyway**
- **In modern OpenGL with VBOs, you can use any type, but you must tell OpenGL the type via glVertexAttribPointer**

Drawing Primitives: Immediate Mode Syntax (1/2)

- **glBegin -> glEnd**

```
glBegin(primitive_type)
    # Set color for subsequent vertices
    glColor3f(r, g, b)
    # Specify vertices
    glVertex2f(x, y)    # 2D vertex
    glVertex3f(x, y, z) # 3D vertex
    # ... more vertices ...
glEnd()
```

Drawing Primitives: Immediate Mode Syntax (2/2)

- **primitive_type**
  - Determines how vertices are assembled
- **Colors set with glColor apply to all vertices until changed**

Drawing Primitives: Examples of Using Different glVertex Variants (1/3)

- 2D float

```
# Using 2D float coordinates
glBegin(GL_POINTS)
glVertex2f(-0.5, -0.5)
glVertex2f( 0.5,  0.5)
glEnd()
```

Drawing Primitives: Examples of Using Different glVertex Variants (2/3)

- 3D float coordinates

```
# Using 3D float coordinates (z=0)
glBegin(GL_LINES)
glVertex3f(-0.8, -0.8, 0.0)
glVertex3f( 0.8,  0.8, 0.0)
glEnd()
```

- Using vector versions

Drawing Primitives: Examples of Using Different glVertex Variants (3/3)

- Using vector versions

```
# Using vector versions
var_points = [(-0.5, 0.5), (0.5, 0.5), (0.5, -0.5), (-0.5, -0.5)]
glBegin(GL_LINE_LOOP)
for var_p in var_points:
    glVertex2fv(var_p)   # pass tuple directly
glEnd()
```

Drawing Primitives: Setting Colors with glColor (1/2)

- In immediate mode, we can set the current color with glColor3f(r, g, b) or glColor4f(r, g, b, a)
- The color applies to all vertices specified after it, until changed

Drawing Primitives: Setting Colors with glColor (2/2)

- Example 1. Produces a triangle with red, green, and blue vertices – colors are interpolated across the triangle

```
glBegin(GL_TRIANGLES)
glColor3f(1.0, 0.0, 0.0)   # red
glVertex2f(-0.5, -0.5)
glColor3f(0.0, 1.0, 0.0)   # green
glVertex2f( 0.5, -0.5)
glColor3f(0.0, 0.0, 1.0)   # blue
glVertex2f( 0.0,  0.5)
glEnd()
```

Drawing Primitives: Our Base Pygame/OpenGL Template (1/5)

```
import pygame
from pygame.locals import *
from OpenGL.GL import *

def fxn_draw_primitives():
    # Drawing code will go here
    pass

def main():
    var_running = True
    var_display = (800, 600)
```

Drawing Primitives: Our Base Pygame/OpenGL Template (2/5)

```
pygame.init()
pygame.display.set_mode(var_display, DOUBLEBUF OPENGL
RESIZABLE)
pygame.display.set_caption("Immediate Mode Demo")
var_clock = pygame.time.Clock()

print("OpenGL version:", glGetString(GL_VERSION).decode())
print("Vendor:", glGetString(GL_VENDOR).decode())
print("Renderer:", glGetString(GL_RENDERER).decode())
```

Drawing Primitives: Our Base Pygame/OpenGL Template (3/5)

```
    while var_running:
        for var_event in pygame.event.get():
            if var_event.type == QUIT:
                var_running = False
            elif var_event.type == KEYDOWN and var_event.key ==
K_ESCAPE:
                var_running = False
            elif var_event.type == VIDEORESIZE:
                var_display = var_event.size
                pygame.display.set_mode(var_display, DOUBLEBUF OPENGL
RESIZABLE)
```

Drawing Primitives: Our Base Pygame/OpenGL Template (4/5)

```
        glViewport(0, 0, var_display[0], var_display[1])

    glClearColor(0.2, 0.3, 0.4, 1.0)
    glClear(GL_COLOR_BUFFER_BIT)

    fxn_draw_primitives()

    pygame.display.flip()
    var_clock.tick(60)
```

Drawing Primitives: Our Base Pygame/OpenGL Template (5/5)

```
    pygame.quit()

if __name__ == "__main__":
    main()
```

Lecture Series Outline

- Recap – The Graphics Pipeline
- Vertex Attributes – What Makes Up a Vertex?
- Understanding the Coordinate System on the Pygame Canvas
- Specifying 3D Vertices in OpenGL (Immediate Mode)
- Drawing Primitives – Points, Lines, and Triangles
- Building Complex Objects – A 3D Cube
- Complete 3D Example with Code Explanation
- Next Steps in the Pipeline
- Code Repository and File List

Drawing Primitives: Drawing Points (GL_POINTS) (1/4)

- Primitive type: GL_POINTS
- Each vertex becomes a single point
- Point size controlled by glPointSize(size)
- Full code available in csc4505_lecture5_points.py (GitHub)

Drawing Primitives: Drawing Points (GL_POINTS) (2/4)

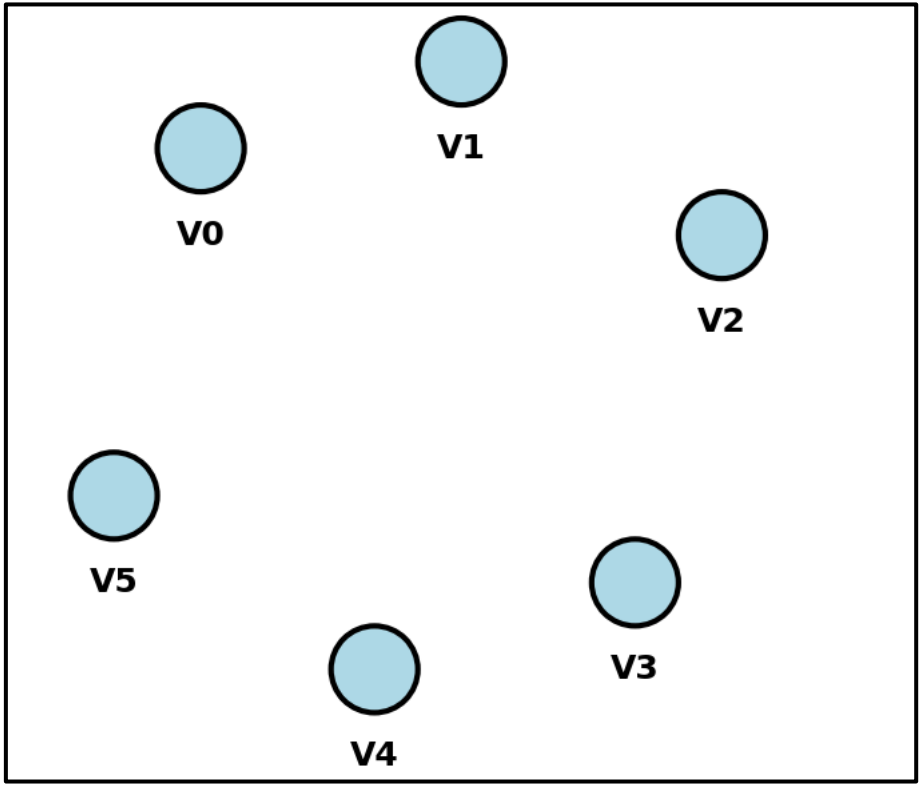
- **Each vertex becomes a single point**
  - Control size with glPointSize()

```
def fxn_draw_primitives():
    # Draw four white points at the corners
    glPointSize(5.0)
    glBegin(GL_POINTS)
    glColor3f(1.0, 1.0, 1.0)  # white
    glVertex2f(-0.8,  0.8)    # top-left
    glVertex2f( 0.8,  0.8)    # top-right
    glVertex2f( 0.8, -0.8)    # bottom-right
    glVertex2f(-0.8, -0.8)    # bottom-left
    glEnd()
```

Drawing Primitives: Drawing Points (GL_POINTS) (3/4)

- **Primitive type: GL_POINTS**
  - glPointSize(5.0): Makes points larger than the default 1 pixel
  - glBegin(GL_POINTS): Tells OpenGL to treat each vertex as an independent point
  - glColor3f(1.0, 1.0, 1.0): Sets the current colour to white; all vertices until the next colour change will be white
  - glVertex3f(...): Each call specifies a vertex at the given NDC coordinates
  - glEnd(): Finishes the point list; the GPU renders the four points
- **Four white points appear at the corners of the window (in screen space after viewport mapping)**

Drawing Primitives: Drawing Points (GL_POINTS) (4/4)



Drawing Primitives: Drawing Lines (GL_LINES) (1/4)

- Primitive type: GL_LINES
- Each pair of vertices forms an independent line segment
- With 4 vertices, we get two lines: (v0,v1) and (v2,v3)
- Line width controlled by glLineWidth(width)
- Full code available in csc4505_lecture5_lines.py (GitHub)

Drawing Primitives: Drawing Lines (GL_LINES) (2/4)

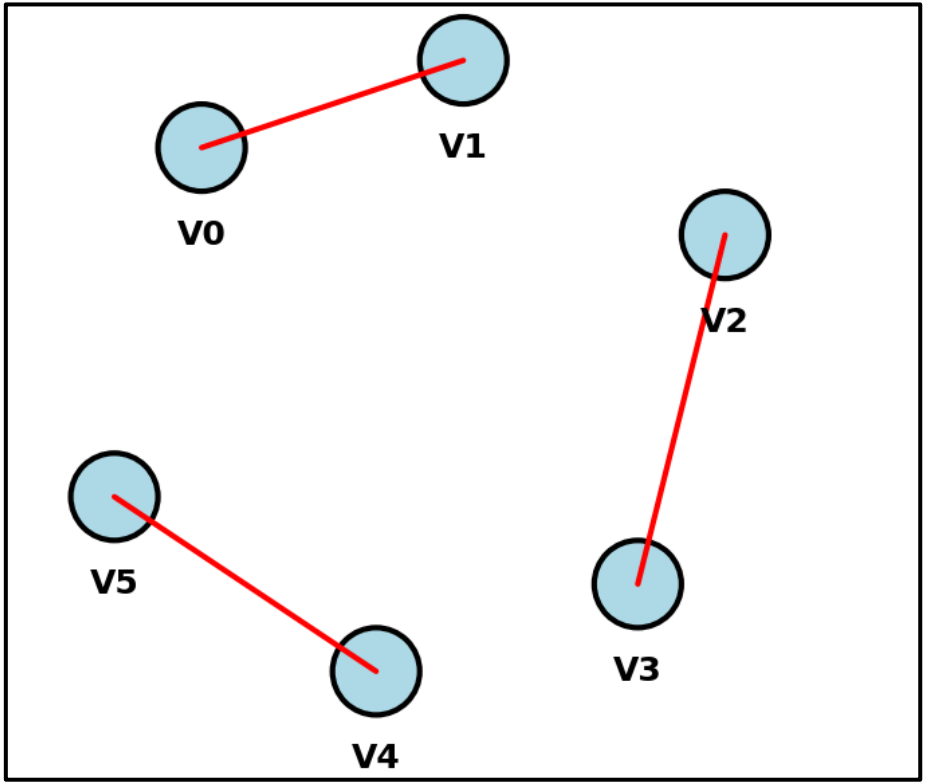
- Each pair of vertices forms an independent line segment

```
def fxn_draw_primitives():
    # Draw two diagonal green lines
    glLineWidth(2.0)
    glBegin(GL_LINES)
    glColor3f(0.0, 1.0, 0.0)  # green
    glVertex2f(-0.5,  0.5)    # start line 1
    glVertex2f( 0.5, -0.5)    # end line 1
    glVertex2f(-0.5, -0.5)    # start line 2
    glVertex2f( 0.5,  0.5)    # end line 2
    glEnd()
```

Drawing Primitives: Drawing Lines (GL_LINES) (3/4)

- **Explanation**
  - glLineWidth(2.0): Makes lines thicker for visibility
  - glBegin(GL_LINES): Each two vertices define a separate line
  - glColor3f(0.0,1.0,0.0): Sets colour to green for all following vertices
  - The first two vertices define one diagonal line from top-left to bottom-right
  - The next two vertices define another diagonal line from bottom-left to top-right
  - glEnd(): Completes the lines; GPU draws two green lines crossing to form an X
  - ***If an odd number of vertices is given, the last one is ignored***

Drawing Primitives: Drawing Lines (GL_LINES) (4/4)



Drawing Primitives: Drawing Line Strips (GL_LINE_STRIP) (1/5)

- Primitive type: GL_LINE_STRIP
- Vertices are connected in sequence: v0–v1, v1–v2, v2–v3, … forming a continuous polyline
- Last vertex does not connect back to first
- Full code available in csc4505_lecture5_line_strip.py (GitHub)

Drawing Primitives: Drawing Line Strips (GL_LINE_STRIP) (2/5)

- Vertices connected in sequence: v0-v1, v1-v2, v2-v3, ...

```
def fxn_draw_primitives(): # Draw a yellow zigzag line
    glBegin(GL_LINE_STRIP)
    glColor3f(1.0, 1.0, 0.0)  # yellow
    glVertex2f(-0.9,  0.2)
    glVertex2f(-0.6,  0.5)
    glVertex2f(-0.3,  0.2)
    glVertex2f( 0.0,  0.5)
    glVertex2f( 0.3,  0.2)
```

Drawing Primitives: Drawing Line Strips (GL_LINE_STRIP) (3/5)

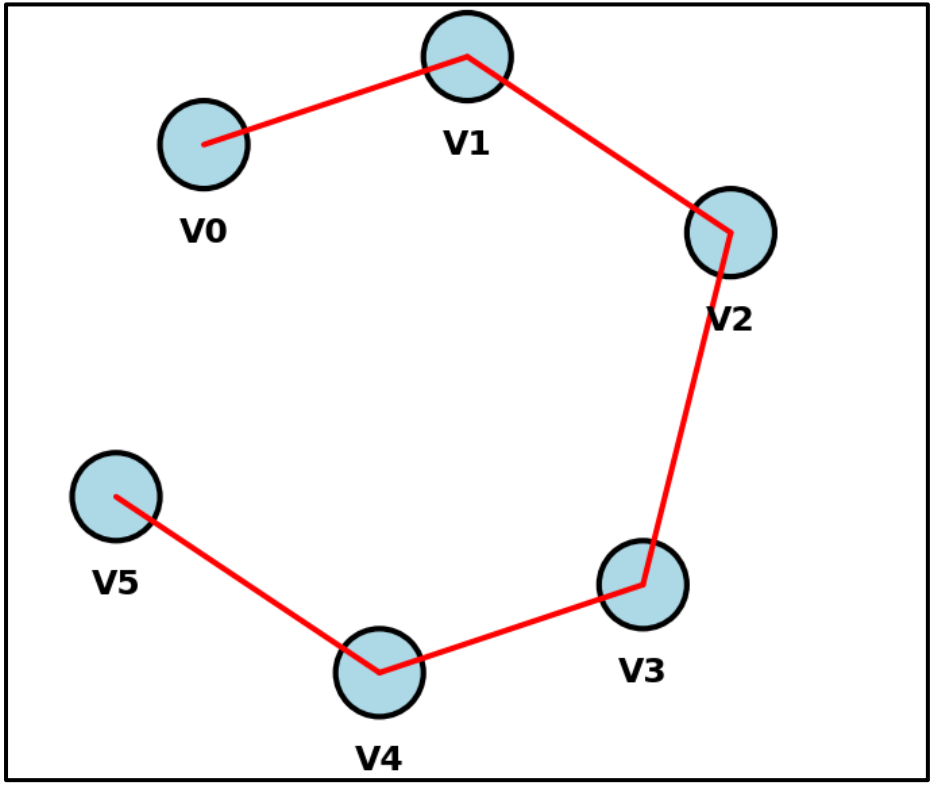
- Vertices connected in sequence: v0-v1, v1-v2, v2-v3, ...

```
def fxn_draw_primitives(): # Draw a yellow zigzag line
    # [...]
    # [...]
    glVertex2f( 0.9,  0.2)
    glEnd()
```

Drawing Primitives: Drawing Line Strips (GL_LINE_STRIP) (4/5)

- **Explanation**
  - glBegin(GL_LINE_STRIP): Vertices will be connected in order
  - glColor3f(1,1,0): Sets colour to yellow for the entire strip
  - Each glVertex3f adds a vertex. OpenGL connects v0–v1, v1–v2, …, v5–v6
  - The result is a continuous zigzag line
- **Example use cases: Drawing polylines, graphs, or any connected sequence**

Drawing Primitives: Drawing Line Strips (GL_LINE_STRIP) (5/5)



Drawing Primitives: Drawing Line Loops (GL_LINE_LOOP) (1/4)

- Primitive type: GL_LINE_LOOP
- Like a line strip, but also connects the last vertex back to the first, forming a closed loop
- Full code available in csc4505_lecture5_line_loop.py (GitHub)

Drawing Primitives: Drawing Line Loops (GL_LINE_LOOP) (2/4)

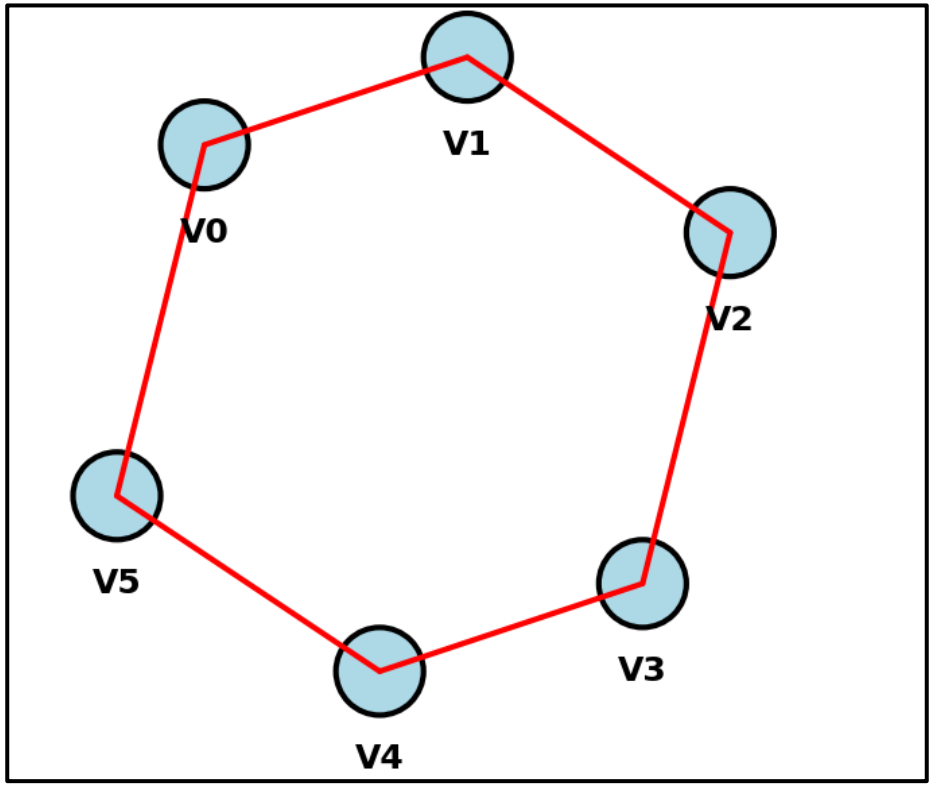
- Like line strip, but also connects last vertex back to first

```
def fxn_draw_primitives():
    # Draw a cyan triangle outline
    glBegin(GL_LINE_LOOP)
    glColor3f(0.0, 1.0, 1.0)  # cyan
    glVertex2f(-0.7, -0.2)
    glVertex2f( 0.7, -0.2)
    glVertex2f( 0.0, -0.7)
    glEnd()
```

Drawing Primitives: Drawing Line Loops (GL_LINE_LOOP) (3/4)

- **Explanation**
  - glBegin(GL_LINE_LOOP): Vertices are connected in order, and then v2 is connected back to v0
  - The three vertices define a triangle outline
- **Example use cases: Drawing wireframe polygons without filling them**

Drawing Primitives: Drawing Line Loops (GL_LINE_LOOP) (4/4)



Drawing Primitives: Drawing Triangles (GL_TRIANGLES) (1/4)

- Primitive type: GL_TRIANGLES
- Each triple of vertices forms an independent, filled triangle
- This is the most important primitive for 3D graphics
- Full code available in csc4505_lecture5_triangles.py (GitHub)

Drawing Primitives: Drawing Triangles (GL_TRIANGLES) (2/4)

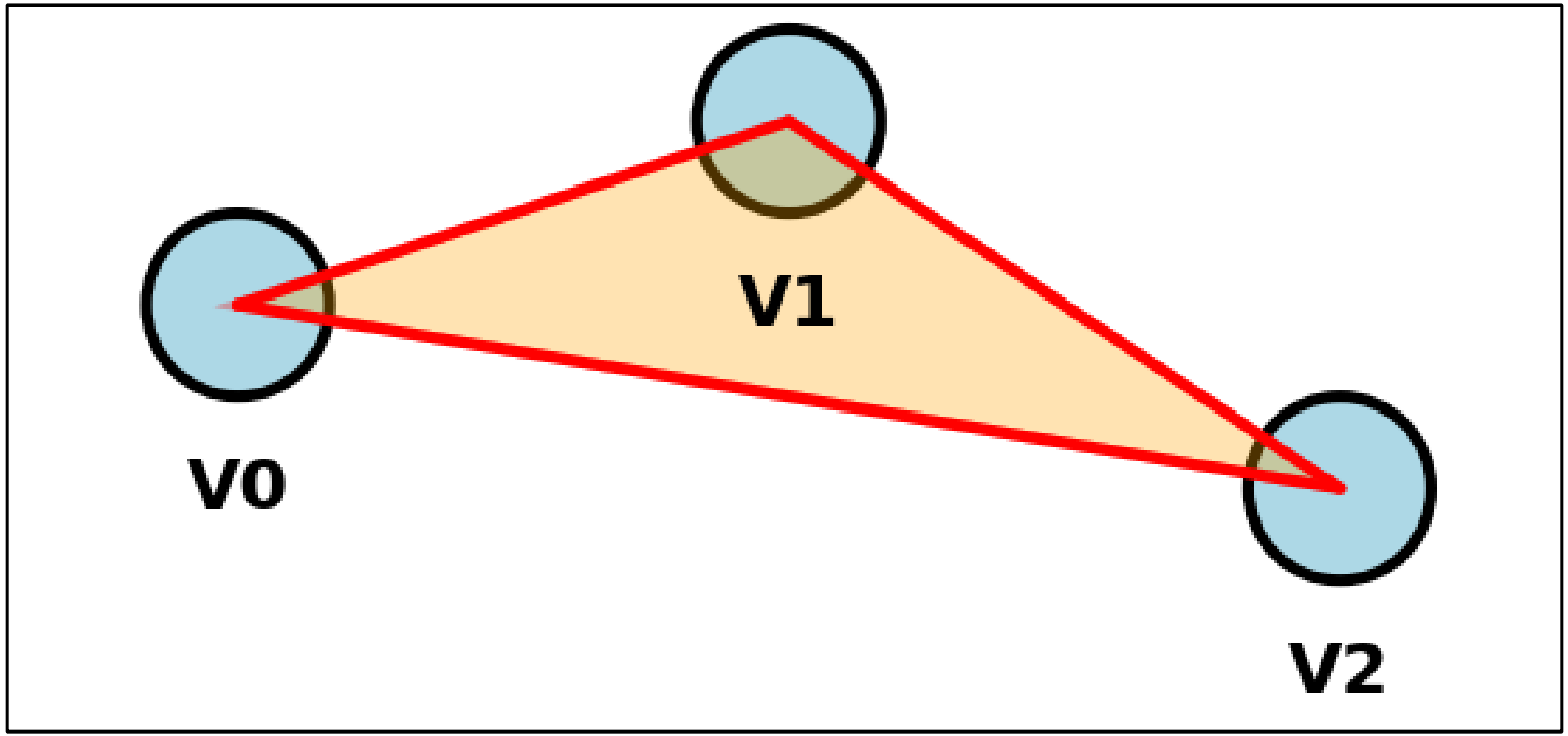
- Each triple of vertices forms a filled triangle

```
def fxn_draw_primitives():
    # Draw a red filled triangle
    glBegin(GL_TRIANGLES)
    glColor3f(1.0, 0.0, 0.0)  # red
    glVertex2f(-0.8, -0.8)
    glVertex2f(-0.2, -0.8)
    glVertex2f(-0.5, -0.3)
    glEnd()
```

Drawing Primitives: Drawing Triangles (GL_TRIANGLES) (3/4)

- **Explanation**
  - glBegin(GL_TRIANGLES): Each three vertices make one triangle
  - We set a different colour before each vertex
  - OpenGL interpolates the colours across the triangle, creating a smooth gradient from red to green to blue
- **Why triangles? Any 3D surface can be approximated by triangles, and the GPU is highly optimised for them**

Drawing Primitives: Drawing Triangles (GL_TRIANGLES) (4/4)



Drawing Primitives: Drawing Triangle Fans (GL_TRIANGLE_FAN) (1/4)

- Primitive type: GL_TRIANGLE_FAN
- The first vertex is the hub. Each subsequent pair forms a triangle with the hub: (v0,v1,v2), (v0,v2,v3), ...
- Efficient for shapes that share a common point (e.g., a filled circle)
- Full code available in csc4505_lecture5_triangle_fan.py (GitHub)

Drawing Primitives: Drawing Triangle Fans (GL_TRIANGLE_FAN) (2/4)

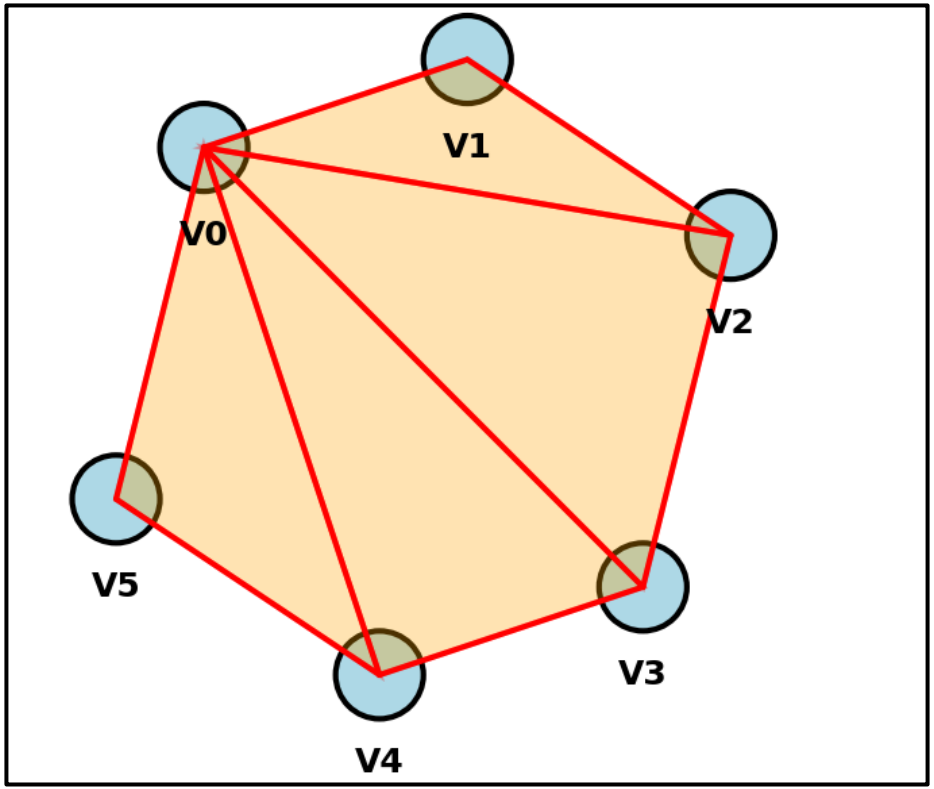
- First vertex is hub; each subsequent pair forms triangle with hub

```
def fxn_draw_primitives():
    # Draw a blue quadrilateral as a triangle fan
    glBegin(GL_TRIANGLE_FAN)
    glColor3f(0.0, 0.0, 1.0)  # blue
    glVertex2f(0.5, -0.5)     # hub
    glVertex2f(0.8, -0.5)     # v1
    glVertex2f(0.8, -0.2)     # v2
    glVertex2f(0.5, -0.2)     # v3
    glEnd()
```

Drawing Primitives: Drawing Triangle Fans (GL_TRIANGLE_FAN) (3/4)

- **Explanation**
  - glBegin(GL_TRIANGLE_FAN): The first vertex (0.5,-0.5) is the hub
  - Triangles formed: (hub,v1,v2) and (hub,v2,v3). This fills a quadrilateral
- **Example use case: Drawing cones, disks, or any shape with a central point**

Drawing Primitives: Drawing Triangle Fans (GL_TRIANGLE_FAN) (4/4)



Drawing Primitives: Drawing Triangle Strips (GL_TRIANGLE_STRIP) (1/4)

- Primitive type: GL_TRIANGLE_STRIP
- Vertices form a strip of triangles: (v0,v1,v2), (v1,v2,v3), (v2,v3,v4), ...
- Very efficient for long, connected surfaces (e.g., terrain strips)
- Full code available in csc4505_lecture5_triangle_strip.py (GitHub)

Drawing Primitives: Drawing Triangle Strips (GL_TRIANGLE_STRIP) (2/4)

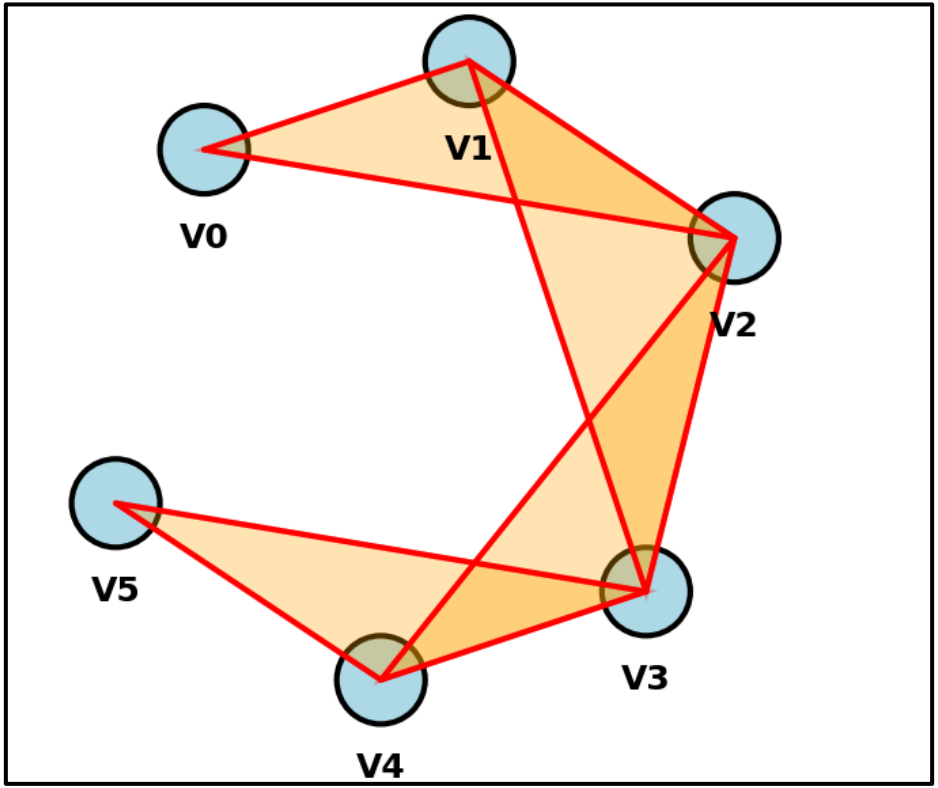
- Forms triangles: (v0,v1,v2), (v1,v2,v3), (v2,v3,v4), ...

```
def fxn_draw_primitives():
    # Draw a magenta shape with triangle strip
    glBegin(GL_TRIANGLE_STRIP)
    glColor3f(1.0, 0.0, 1.0)  # magenta
    glVertex2f(0.2, 0.2)      # v0
    glVertex2f(0.5, 0.5)      # v1
    glVertex2f(0.5, 0.2)      # v2
    glVertex2f(0.8, 0.5)      # v3
    glVertex2f(0.8, 0.2)      # v4
    glEnd()
```

Drawing Primitives: Drawing Triangle Strips (GL_TRIANGLE_STRIP) (3/4)

- **Explanation**
  - glBegin(GL_TRIANGLE_STRIP): Triangles are (v0,v1,v2), (v1,v2,v3), (v2,v3,v4)
  - With 5 vertices, we get 3 triangles forming a shape
- **Example use cases: Rendering height maps, ribbons, or any long mesh**

Drawing Primitives: Drawing Triangle Strips (GL_TRIANGLE_STRIP) (4/4)

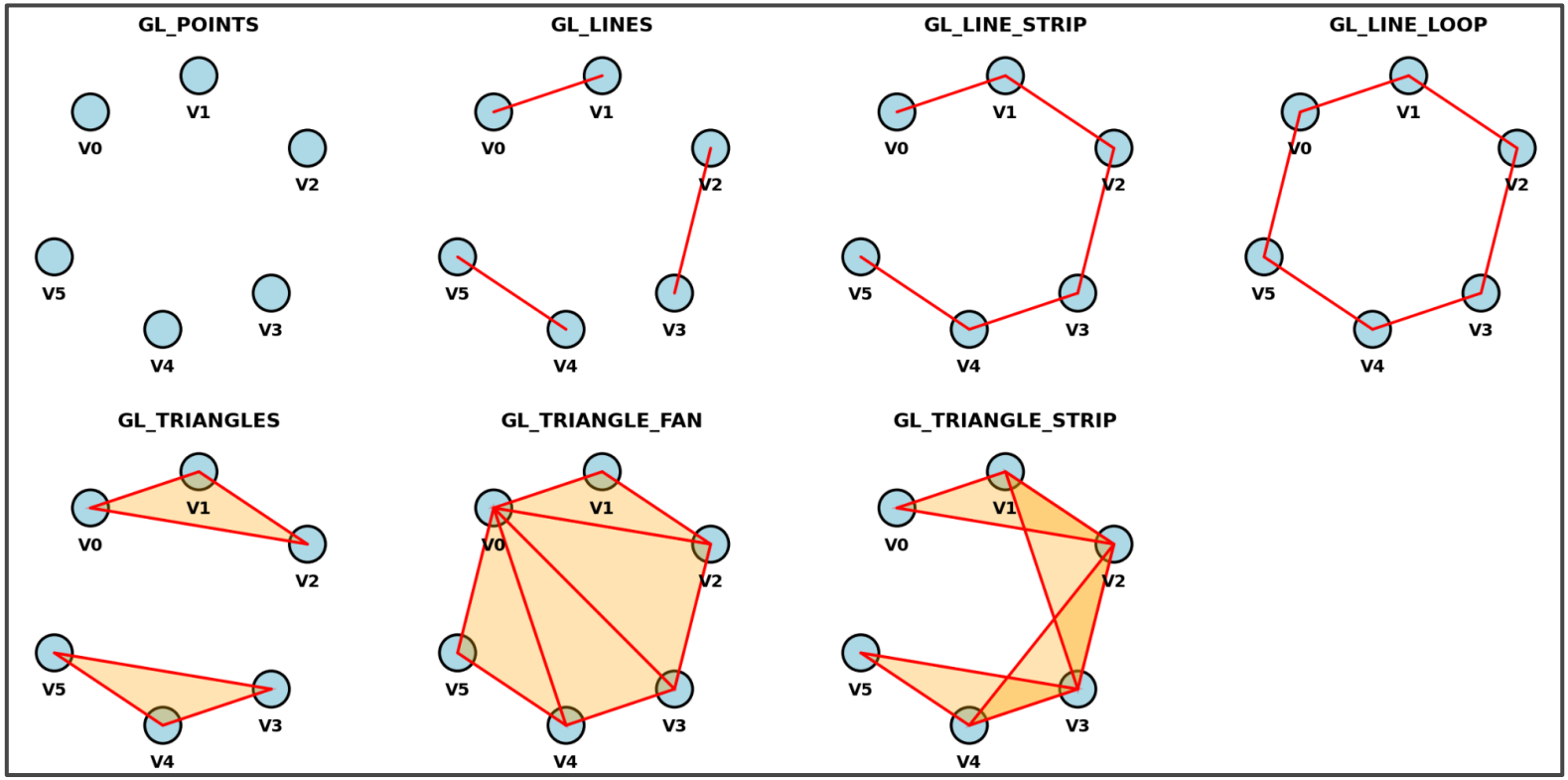


Drawing Primitives: Complete Immediate Mode Drawing Function (1/2)

- Complete fxn_draw_primitives() with all examples combined

  # See CSC 4505 github repository

Drawing Primitives: Complete Immediate Mode Drawing Function (2/2)



Lecture Series Outline

- Recap – The Graphics Pipeline
- Vertex Attributes – What Makes Up a Vertex?
- Understanding the Coordinate System on the Pygame Canvas
- Specifying 3D Vertices in OpenGL (Immediate Mode)
- Drawing Primitives – Points, Lines, and Triangles
- Building Complex Objects – A 3D Cube
- Complete 3D Example with Code Explanation
- Next Steps in the Pipeline
- Code Repository and File List

Drawing Primitives: Building a 3D Cube (1/22)

- **We will now construct a 3D cube using the immediate mode drawing commands we have learned**
  - The cube will be centered at the origin (0,0,0) and have side length 1 (so each side extends 0.5 units from the center)
  - We will draw each face as two triangles, giving us 12 triangles in total
  - Why a cube?
    - The cube is the simplest 3D object and demonstrates how to combine multiple triangles to form a solid shape
    - It also introduces the concept of vertices and indices, which are fundamental in modern graphics

Drawing Primitives: Building a 3D Cube (2/22)

- **We will now construct a 3D cube using the immediate mode drawing commands we have learned**
  - Let's begin by defining the 8 vertices of the cube
    - We store the vertex data in a NumPy array
    - Each row contains [x, y, z, r, g, b] – the position and colour for one vertex

Drawing Primitives: Building a 3D Cube (3/22)

- Cube vertices (centered at origin, side length 1)

```
import numpy as np
var_cube_vertices = np.array([
  [-0.5, -0.5, -0.5, 0.0, 1.0, 0.0], # V0 (back face, green)
  [ 0.5, -0.5, -0.5, 0.0, 1.0, 0.0], # V1
  [ 0.5,  0.5, -0.5, 0.0, 1.0, 0.0], # V2
  [-0.5,  0.5, -0.5, 0.0, 1.0, 0.0], # V3
  [-0.5, -0.5,  0.5, 1.0, 0.0, 0.0], # V4 (front face, red)
  [ 0.5, -0.5,  0.5, 1.0, 0.0, 0.0],   # V5
  [ 0.5,  0.5,  0.5, 1.0, 0.0, 0.0],   # V6
  [-0.5,  0.5,  0.5, 1.0, 0.0, 0.0]    # V7
], dtype=np.float32)
```

Drawing Primitives: Building a 3D Cube (3/22)

- **Numpy explanation**
  - np.array(...) creates a multi-dimensional array. The outer brackets enclose the rows, and each row is a list of 6 numbers
  - dtype=np.float32 ensures each number is stored as a 32-bit float, which is the standard for graphics data
  - The array has shape (8, 6) – 8 vertices, each with 6 attributes
  - Accessing a row: var_cube_vertices[3] gives the 4th vertex (indexing starts at 0)
  - Accessing a component: var_cube_vertices[3, 0] gives the x-coordinate of vertex 3

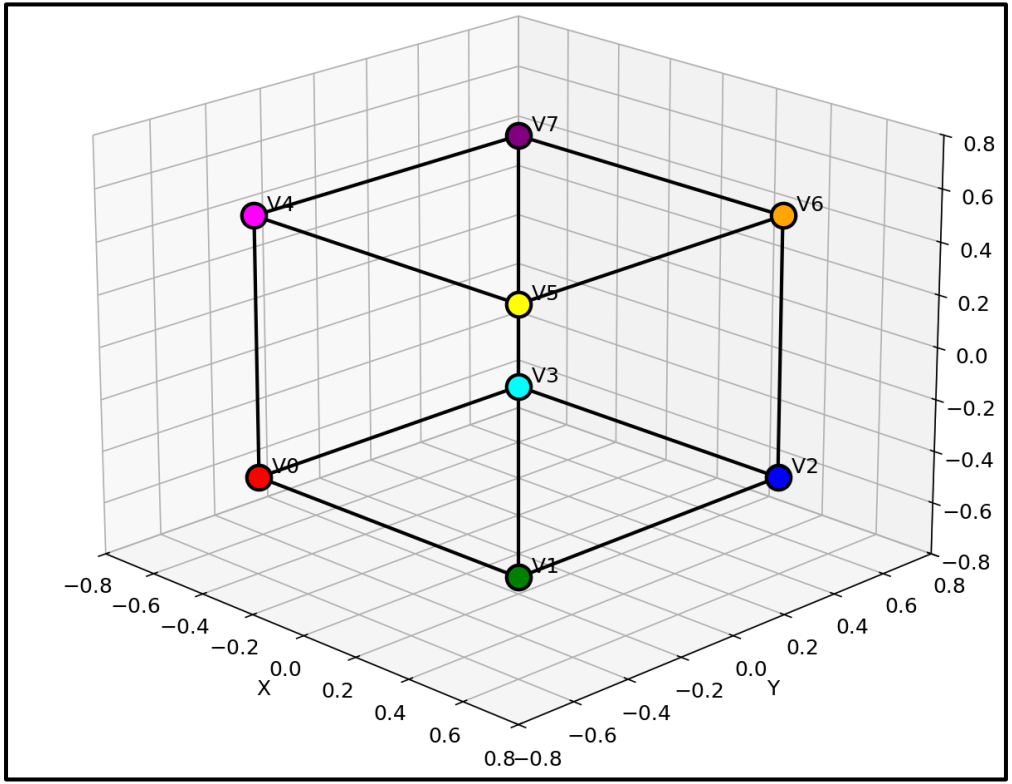
Drawing Primitives: Building a 3D Cube (4/22)

- **What do the coordinates mean?**
  - x: left/right. -0.5 = left side, +0.5 = right side
  - y: bottom/top. -0.5 = bottom, +0.5 = top
  - z: back/front. -0.5 = back (farther from viewer), +0.5 = front (closer to viewer)

Drawing Primitives: Building a 3D Cube (5/22)

- **Thus**
  - V0 is at the back, bottom, left corner
  - V1 is back, bottom, right
  - V2 is back, top, right
  - V3 is back, top, left
  - V4 is front, bottom, left
  - V5 is front, bottom, right
  - V6 is front, top, right
  - V7 is front, top, left

Drawing Primitives: Building a 3D Cube (6/22)



Drawing Primitives: Building a 3D Cube (7/22)

- **Why Different Colours for Front and Back?**
  - Notice that the back-face vertices (V0–V3) are all green, while the front-face vertices (V4–V7) are all red
  - This is just to make the two sets visually distinct
  - Later, when we draw the cube, we will assign per-face colours so each face has its own colour
  - The vertex colours here are not used directly; they are only for illustration in the diagram

Drawing Primitives: Building a 3D Cube (8/22)

- **Defining the Triangles (Indices)**
  - We cannot simply draw the vertices; we must tell OpenGL how to connect them to form triangles. We do this with an index array
  - Each triangle is defined by three vertex indices

Drawing Primitives: Building a 3D Cube (9/22)

- **Defining the Triangles (Indices)**

```
var_cube_indices = np.array([
    # Front face (red) – two triangles
    [4,5,6], [4,6,7],
    # Back face (green) – two triangles
    [0,2,1], [0,3,2],
    # Left face (blue)
    [0,3,7], [0,7,4],
    # Right face (yellow)
    [1,5,6], [1,6,2],
    # [...]
```

Drawing Primitives: Building a 3D Cube (10/22)

- **Defining the Triangles (Indices)**

```
        # Top face (cyan)
        [3,2,6], [3,6,7],
        # Bottom face (magenta)
        [0,1,5], [0,5,4]
    ], dtype=np.int32)
```

Drawing Primitives: Building a 3D Cube (11/22)

- **NumPy explanations**
  - This array has shape (12, 3) – 12 triangles, each with 3 vertex indices
  - dtype=np.int32 stores the indices as 32-bit integers, which is what OpenGL expects for element indices
  - Each row is a list of three integers that refer to the rows of var_cube_vertices

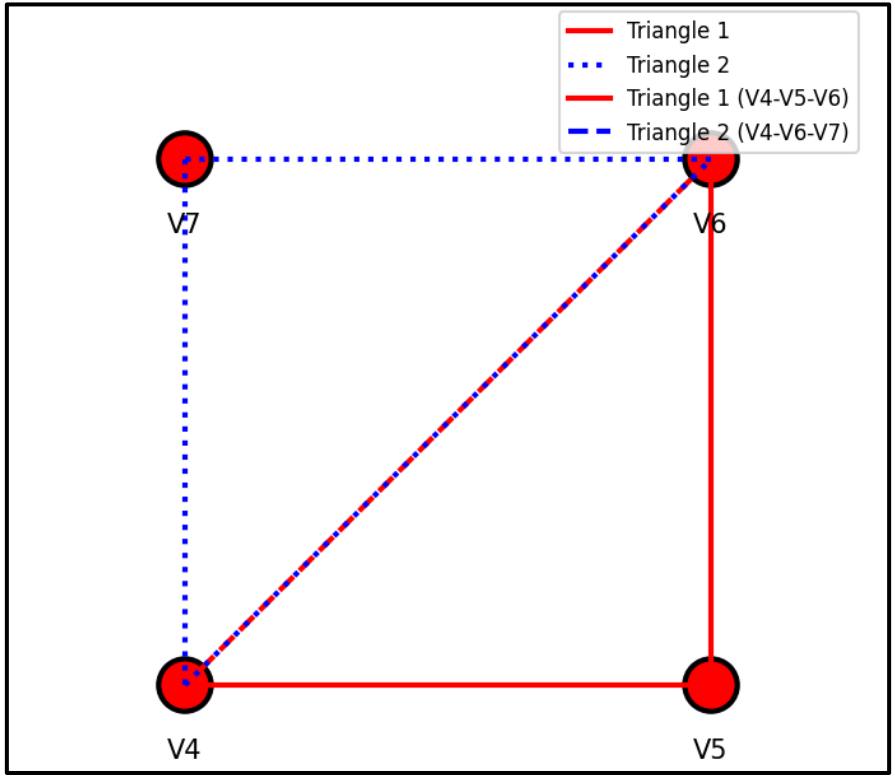
Drawing Primitives: Building a 3D Cube (12/22)

- **How the indices work**
  - For the front face (z = +0.5), we use vertices V4, V5, V6, V7. The two triangles are (V4, V5, V6) and (V4, V6, V7). This covers the square
  - For the back face (z = -0.5), we use V0, V1, V2, V3. The triangles are (V0, V2, V1) and (V0, V3, V2). Notice the order is chosen to be counter-clockwise when viewed from outside
  - The left face (x = -0.5) uses V0, V3, V7, V4: triangles (V0, V3, V7) and (V0, V7, V4)
  - Right face (x = +0.5) uses V1, V2, V6, V5: triangles (V1, V5, V6) and (V1, V6, V2)

Drawing Primitives: Building a 3D Cube (13/22)

- **How the indices work**
  - Top face (y = +0.5) uses V3, V2, V6, V7: triangles (V3, V2, V6) and (V3, V6, V7)
  - Bottom face (y = -0.5) uses V0, V1, V5, V4: triangles (V0, V1, V5) and (V0, V5, V4)

Drawing Primitives: Building a 3D Cube (14/22)



Drawing Primitives: Building a 3D Cube (15/22)

- To make each face a different colour, we define an array of colours for the six faces

```
var_face_colours = [
    [1.0, 0.0, 0.0],   # red   – front
    [0.0, 1.0, 0.0],   # green – back
    [0.0, 0.0, 1.0],   # blue  – left
    [1.0, 1.0, 0.0],   # yellow – right
    [0.0, 1.0, 1.0],   # cyan  – top
    [1.0, 0.0, 1.0]    # magenta – bottom
]
```

Drawing Primitives: Building a 3D Cube (16/22)

- **To make each face a different colour, we define an array of colours for the six faces**
  - This is a Python list of lists
    - Each inner list contains three floats (r,g,b) for that face
  - The order of faces must match the order of the 12 triangles in the index array
    - First two triangles = front, next two = back, etc

Drawing Primitives: Building a 3D Cube (17/22)

- Now we write the drawing function. We loop over the six faces, set the colour, then loop over the two triangles of that face, and finally loop over the three vertices of each triangle, fetching the coordinates from var_cube_vertices

Drawing Primitives: Building a 3D Cube (18/22)

- **Code**

```
glBegin(GL_TRIANGLES)
for var_face in range(6):  # for each face (0..5)
    glColor3f(*var_face_colours[var_face]) # set the face colour
    for var_i in range(2):  # two triangles per face
        var_tri = var_cube_indices[var_face*2 + var_i] # get indices
        for var_j in range(3): # three vertices per triangle
            var_v = var_cube_vertices[var_tri[var_j]] # fetch vertex
            glVertex3f(var_v[0], var_v[1], var_v[2]) # pass x,y,z
glEnd()
```

Drawing Primitives: Building a 3D Cube (19/22)

- **Step-by-step explanation**
  - glBegin(GL_TRIANGLES): we are about to send a sequence of triangles
  - for var_face in range(6): we have six faces
  - glColor3f(*var_face_colours[var_face]): set the current colour to the colour of this face. The * unpacks the list into three arguments
  - for var_i in range(2): each face consists of two triangles
  - var_tri = var_cube_indices[var_face*2 + var_i]: each face contributes two consecutive rows in the index array. For face 0, we take rows 0 and 1; for face 1, rows 2 and 3; etc
  - for var_j in range(3): each triangle has three vertices

Drawing Primitives: Building a 3D Cube (20/22)

- **Step-by-step explanation**
  - var_v = var_cube_vertices[var_tri[var_j]]: get the row of var_cube_vertices corresponding to the vertex index. This row contains [x,y,z,r,g,b]
  - glVertex3f(var_v[0], var_v[1], var_v[2]): pass only the position to OpenGL (we already set the colour earlier)
  - After all loops, glEnd() finishes the primitive

Drawing Primitives: Building a 3D Cube (21/22)

- **Why this approach?**
  - It separates data (vertices, indices, colours) from drawing logic, which is exactly how modern OpenGL works with buffer objects
  - It also demonstrates how NumPy arrays can be used to store and index vertex data efficiently

Drawing Primitives: Building a 3D Cube (22/22)

- We can encapsulate the cube drawing in a function
  - Function can be called in main loop post set-up of transformations

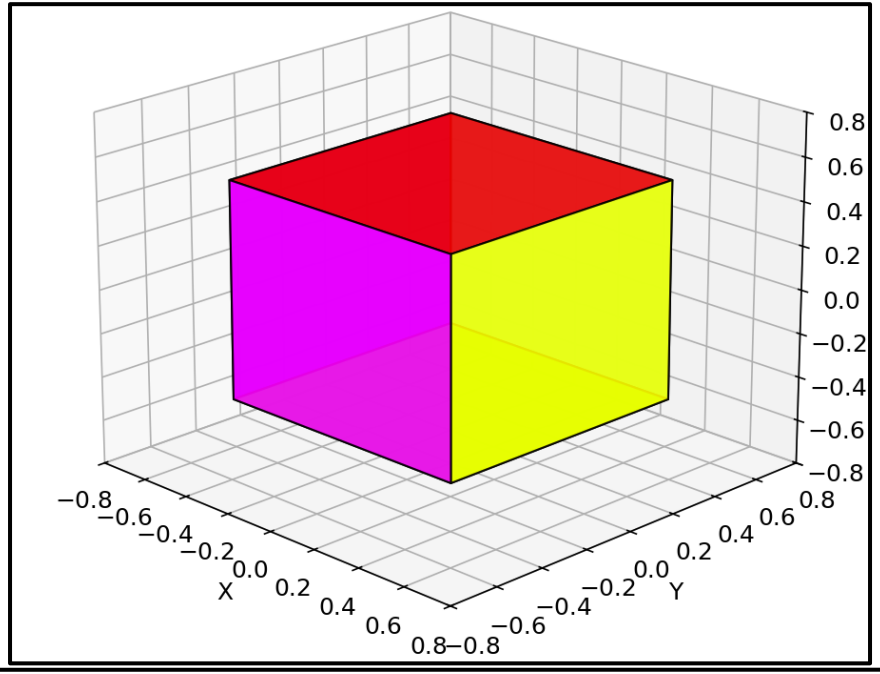
```
def fxn_draw_cube():
    glBegin(GL_TRIANGLES)
    for var_face in range(6):
        glColor3f(*var_face_colours[var_face])
        for var_i in range(2):
            var_tri = var_cube_indices[var_face*2 + var_i]
            for var_j in range(3):
                var_v = var_cube_vertices[var_tri[var_j]]
                glVertex3f(var_v[0], var_v[1], var_v[2])
    glEnd()
```

Drawing Primitives: Building a 3D Cube—Running the Cube& Final Result (1/4)

- When we run the complete cube code (with depth testing and rotation enabled), we see a smoothly rotating cube where each face has a distinct colour

Drawing Primitives: Building a 3D Cube—Running the Cube& Final Result (2/4)

- Image below is simulation of what is seen on your screen



Drawing Primitives: Building a 3D Cube—Running the Cube& Final Result (3/4)

- **What does this image show?**
  - The cube is centered at the origin
  - Each face is a different colour: front (red), back (green), left (blue), right (yellow), top (cyan), bottom (magenta)
  - Depth testing ensures that faces closer to the viewer hide those behind them
  - The rotation (around a diagonal axis) reveals all six faces, making the 3D structure clear

Drawing Primitives: Building a 3D Cube—Running the Cube& Final Result (4/4)

- **How does this relate to our code?**
  - The vertex data (var_cube_vertices) defines the 8 corners
  - The index array (var_cube_indices) specifies the 12 triangles
  - The per-face colours (var_face_colours) are set inside the drawing loop
  - The rotation is achieved with glRotatef in the modelview matrix
  - This simple example contains all the core ideas of vertex specification
    - Attributes, primitive assembly, and basic transformations
    - It also prepares us for the next lecture, where we will study these transformations in depth

Drawing Primitives: Building a 3D Cube—Alternative Triangle Modes

- **Drawing a cube with GL_TRIANGLES requires 12 triangles (2 per face × 6 faces), each with 3 vertices → 36 vertex calls**
  - Using triangle fans or strips can reduce the count
  - For example, the left face could be drawn as a fan with 4 vertices instead of 6
  - NumPy makes it easy to re-organise data for different primitive types
- ***Homework/Lab***
  - ***Implement rotating cube using fans and strips***

Drawing Primitives: Building a 3D Cube—Rotation via Transformations (1/2)

- **In the cube example, we added a rotation to make the 3D structure visible**
  - This is our first encounter with transformations – operations that move, rotate, or scale objects in 3D space
  - The code responsible for rotation:

```
# Select the modelview matrix stack
glMatrixMode(GL_MODELVIEW)
glLoadIdentity()  # Reset to identity (no transformations)
# Rotate by var_angle degrees around axis (1,1,0)
glRotatef(var_angle, 1, 1, 0)
var_angle += 1.0    # Increase angle each frame
```

Drawing Primitives: Building a 3D Cube—Rotation via Transformations (2/2)

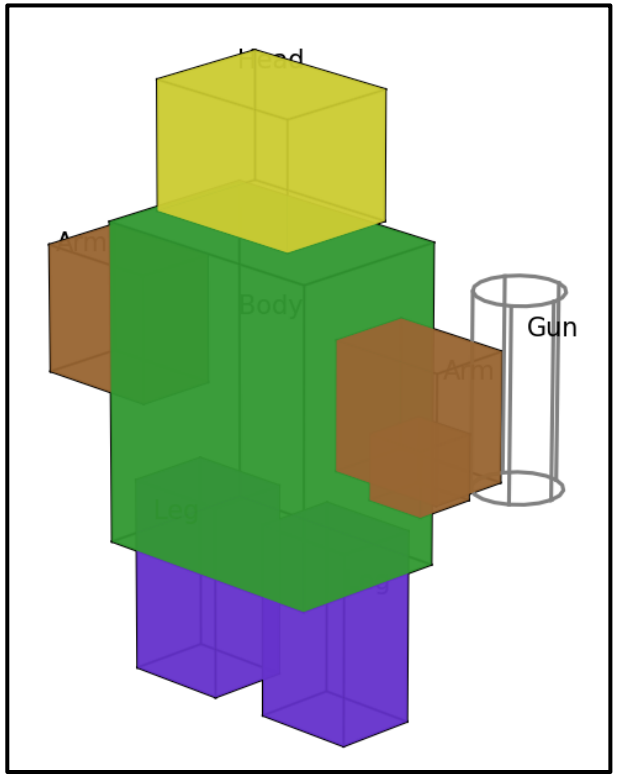
- **We will cover all affine transformations in detail:**
  - Translation, rotation, scaling (uniform and non-uniform)
  - Shearing and reflection
  - Combining transformations using matrix multiplication
  - The difference between model space, world space, view space, and clip space
  - How to build your own transformation matrices and apply them (in both legacy and modern OpenGL)
  - Note: In modern OpenGL (with shaders), transformations are passed as uniform matrices to the vertex shader
    - The mathematical concepts are identical – only the mechanism changes

Drawing Primitives: Building a Polyfield Soldier from Primitives (1/2)

- Now that we know how to draw points, lines, and triangles, let's see how these simple building blocks can be combined to create a recognizable character – a Polyfield soldier
- This demonstrates that any complex 3D shape can be decomposed into basic primitives

Drawing Primitives: Building a Polyfield Soldier from Primitives (2/2)

- **Homework**
  - Draw the replica Polyfield Soldier in the image illustrated on the current slide



Lecture Series Outline

- Recap – The Graphics Pipeline
- Vertex Attributes – What Makes Up a Vertex?
- Understanding the Coordinate System on the Pygame Canvas
- Specifying 3D Vertices in OpenGL (Immediate Mode)
- Drawing Primitives – Points, Lines, and Triangles
- Building Complex Objects – A 3D Cube
- Complete 3D Example with Code Explanation
- Next Steps in the Pipeline
- Code Repository and File List

Next Steps in the Pipeline

- **Now that we can specify vertices and draw 3D objects, the next stage is Vertex Processing**
- **Upcoming lectures**
  - Transformations (translation, rotation, scaling) – moving objects in the scene
  - Viewing – setting up a virtual camera
  - Projection – creating perspective and orthographic views
  - Clipping – removing parts of objects outside the view volume
- **Then we shall study rasterization algorithms (line drawing, circle drawing, polygon filling) – the bridge between vertex specification and the final image**

Lecture Series Outline

- Recap – The Graphics Pipeline
- Vertex Attributes – What Makes Up a Vertex?
- Understanding the Coordinate System on the Pygame Canvas
- Specifying 3D Vertices in OpenGL (Immediate Mode)
- Drawing Primitives – Points, Lines, and Triangles
- Building Complex Objects – A 3D Cube
- Complete 3D Example with Code Explanation
- Next Steps in the Pipeline
- Code Repository and File List

Code Repository and File List (1/2)

- All code examples from this lecture are available on GitHub

Module	Contents
csc4505_lecture5_template.py	Base Pygame/OpenGL window template
csc4505_lecture5_points.py	GL_POINTS example – four white points
csc4505_lecture5_lines.py	GL_LINES example – two green diagonals
csc4505_lecture5_line_strip.py	GL_LINE_STRIP example – yellow zigzag
csc4505_lecture5_line_loop.py	GL_LINE_LOOP example – cyan triangle outline
csc4505_lecture5_triangles.py	GL_TRIANGLES example – coloured triangle
csc4505_lecture5_triangle_fan.py	GL_TRIANGLE_FAN example – blue quadrilateral
csc4505_lecture5_triangle_strip.py	GL_TRIANGLE_STRIP example – magenta shape

Code Repository and File List (2/2)

- All code examples from this lecture are available on GitHub

Module	Contents
csc4505_lecture5_cube.py	3D cube with coloured faces
csc4505_lecture5_all_primitives.py	All primitives combined

Q & A Session

- Comments, concerns and complaints?

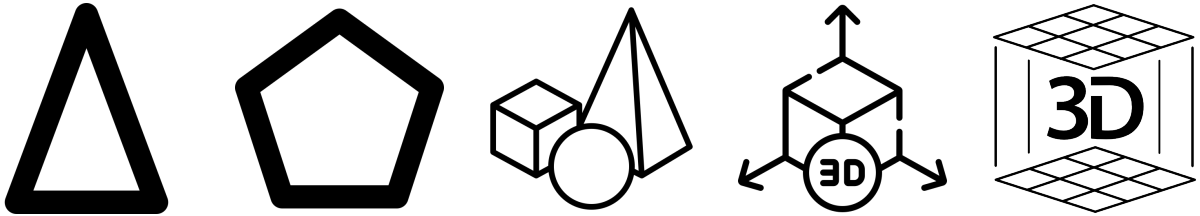
Bibliography

1. **Khronos OpenGL Wiki**
   https://www.khronos.org/opengl/wiki
2. **OpenGL Registry**
   ttps://www.khronos.org/registry/OpenGL
3. **Learn OpenGL**
   https://learnopengl.com
4. **Python & OpenGL for Scientific Visualization**
   https://www.labri.fr/perso/nrougier/python-opengl

csc4505@unza.zm

https://moodle.unza.zm/course/view.php?id=626

YouTube Playlist ID: PLubb3AIPdPhYGt3ax8orjiDo3cuK0tGfx



CSC 4505 (2025/26)
Graphics and Visual Computing

Lecture 5: Vertex Specification

Lighton Phiri <lighton.phiri@unza.zm>
Department of Computing and Informatics
University of Zambia
https://datalab.unza.zm/people/lightonphiri